

Đảm bảo tính tin cậy của các formulation do LLM sinh ra trong bài toán tối ưu hoá

Bản đồ state of the art, khoảng trống nghiên cứu và đề cương phương pháp khả thi

Bản brainstorming cho đề tài nghiên cứu

Cập nhật đến ngày 31/07/2026

Tóm tắt nội dung

Tài liệu này xây dựng một bản đồ bằng chứng có chọn lọc về việc sinh mô hình tối ưu từ ngôn ngữ tự nhiên bằng mô hình ngôn ngữ lớn (LLM), với trọng tâm là *độ tin cậy của formulation* thay vì chỉ khả năng chạy code. Tài liệu phân biệt sáu tầng đảm bảo: hợp lệ cú pháp, khả năng giải, tính khả thi, đúng giá trị nghiệm, trung thành ngữ nghĩa, và độ tin cậy có hiệu chỉnh khi triển khai. Tổng quan cho thấy state of the art đã chuyển từ generate-and-run sang biểu diễn trung gian, huấn luyện có solver, kiểm tra hành vi, kiểm tra hai phía, mutation testing và falsification dựa trên lý thuyết tối ưu. Vì vậy, đóng góp “dùng solver perturbation để kiểm tra formulation” không còn đủ mới ở thời điểm hiện tại. Khoảng trống khả thi hơn là biến các bằng chứng lỗi có cấu trúc thành sửa lỗi cục bộ không regression, đồng thời hiệu chỉnh quyết định accept–repair–abstain. Từ đó, tài liệu đề xuất **CertOpt**: một pipeline *certificate-guided selective repair* trên MILP, kèm câu hỏi nghiên cứu, giả thuyết, thiết kế thí nghiệm, ablation, rủi ro và lộ trình triển khai.

Kết luận chính

Nếu chỉ chọn một hướng để viết paper, lựa chọn được khuyến nghị là:

CertOpt: certificate-guided localized repair + calibrated abstention cho MILP

Đóng góp nên được định vị là *sửa lỗi và ra quyết định có kiểm soát dựa trên certificate*, không phải phát minh lại behavioral verification.

Mục lục

1	Mục tiêu, phạm vi và cách đọc tài liệu	3
1.1	Đối tượng nghiên cứu	3
1.2	Loại tổng quan	3
2	Độ tin cậy của formulation gồm những gì?	3
3	Bản đồ state of the art	4
3.1	Giai đoạn 1: Từ semantic parsing đến agent sinh code	4
3.2	Giai đoạn 2: Dữ liệu có cấu trúc và post-training có solver	4
3.3	Giai đoạn 3: Intermediate representation và validation theo stage	4
3.4	Giai đoạn 4: Verification hướng tới silent failure	4
3.5	Ma trận các công trình tiêu biểu	5

4	Những kết luận đã đủ mạnh để làm nền cho paper	6
5	Khoảng trống nghiên cứu còn mở	6
6	Đề xuất chính: CertOpt	6
6.1	Định nghĩa bài toán	8
6.2	Kiến trúc	8
6.3	Bốn thành phần tạo novelty	8
6.4	Thuật toán ở mức paper	10
6.5	Đóng góp có thể claim	10
7	Câu hỏi nghiên cứu và giả thuyết	10
8	Thiết kế thí nghiệm	11
8.1	Datasets	11
8.2	Benchmark lỗi đề xuất	11
8.3	Ground truth và oracle	11
8.4	Baselines	12
8.5	Metrics	12
8.6	Ablation bắt buộc	12
9	Kết quả nào sẽ làm paper thuyết phục?	12
10	Phạm vi MVP và các hướng thay thế	13
10.1	MVP được khuyến nghị	13
10.2	Hướng thay thế A: Nonlinear/Stochastic Falsification	13
10.3	Hướng thay thế B: Ambiguity-Aware Interactive Modeling	14
10.4	Hướng thay thế C: Certificate-Derived RL Reward	14
11	Rủi ro và cách giảm thiểu	14
12	Lộ trình 12 tuần	14
13	Khung paper có thể dùng ngay	15
14	Quyết định nghiên cứu được khuyến nghị	15

1 Mục tiêu, phạm vi và cách đọc tài liệu

1.1 Đối tượng nghiên cứu

Cho mô tả tự nhiên p , một hệ thống sinh ra formulation

$$M = (X, D, f, C, \theta),$$

trong đó X là biến quyết định, D là miền biến, f là hàm mục tiêu, C là tập ràng buộc và θ là dữ liệu/tham số. Bài toán không chỉ là làm cho solver trả về trạng thái **optimal**; bài toán thực sự là bảo đảm M biểu diễn đúng ý định trong p , và hệ thống biết khi nào bằng chứng hiện có chưa đủ để tin.

Phạm vi chính của đề xuất là LP/MILP với mô tả tiếng Anh hoặc tiếng Việt đã chứa đủ dữ kiện số. Nonlinear, stochastic và multi-objective được giữ làm hướng mở vì chúng làm tăng đáng kể phần lý thuyết và chi phí benchmark.

1.2 Loại tổng quan

Đây là **rapid evidence map** phục vụ chọn đề tài, không phải systematic literature review theo PRISMA. Nguồn được ưu tiên là trang paper chính thức của ACL Anthology, PMLR, NeurIPS/OpenReview và arXiv. Mốc chốt tìm kiếm là 31/07/2026. Các kết quả preprint được ghi rõ để tránh đánh đồng với công trình đã qua phân biện.

Do benchmark, base model, solver và tolerance khác nhau giữa các bài, cụm từ “state of the art” trong tài liệu này chỉ có nghĩa là *frontier method theo từng cơ chế đảm bảo*; không nên xếp hạng tuyệt đối chỉ từ accuracy do các bài tự báo cáo. Khảo sát gần đây còn phát hiện tỷ lệ lỗi đáng kể trong nhiều benchmark, làm cho so sánh chéo càng dễ sai lệch (Xiao et al. 2025).

2 Độ tin cậy của formulation gồm những gì?

Một nguồn nhầm lẫn phổ biến là dùng các khái niệm sau như thể tương đương. Chúng tạo thành các công kiểm tra khác nhau.

Bảng 1: Taxonomy sáu tầng độ tin cậy. Tầng cao không tự động suy ra từ tầng thấp.

Tầng	Câu hỏi kiểm tra	Ví dụ failure vẫn lọt qua
L1. Cú pháp	Code có parse/compile không?	Sai dấu bất đẳng thức nhưng code hợp lệ.
L2. Khả năng giải	Solver có kết thúc với trạng thái hợp lệ không?	Mô hình bỏ một ràng buộc nên giải nhanh hơn.
L3. Feasibility	Nghiem có thỏa chính mô hình đã sinh không?	Nghiem feasible cho mô hình sai, nhưng vi phạm yêu cầu gốc.
L4. Outcome agreement	Objective/solution có khớp nhãn trên một instance không?	Hai feasible set khác nhau tình cờ có cùng optimum.
L5. Semantic faithfulness	Biến, miền, objective và mọi requirement có được biểu diễn đúng không?	Thiếu time window, sai hệ số, sai indexing hoặc sai Big- M .
L6. Deployment reliability	Khi nào hệ thống nên accept, repair, hỏi lại hay abstain?	Verifier không phát hiện lỗi nhưng hệ thống vẫn tự tin deploy.

ReLoop báo cáo một “feasibility–correctness gap” có thể đạt 90 điểm phần trăm trên bài toán compositional, minh họa rõ ràng L2–L3 không đại diện cho L5 (Lian et al. 2026). OptArgus cũng chỉ ra rằng ngay trong nhóm kết quả khớp objective, vẫn có các artifact sai cấu trúc (Z. Li et al. 2026a). Do đó, paper mới phải báo cáo riêng từng tầng và không dùng “solver says optimal” như một chứng nhận ngữ nghĩa.

3 Bản đồ state of the art

3.1 Giai đoạn 1: Từ semantic parsing đến agent sinh code

NL4Opt đặt nền móng bằng cách tách nhận diện thực thể và sinh logical form, sau đó chuyển sang định dạng solver (Ramamonjison et al. 2022; Ramamonjison et al. 2023). Các hệ thống tiếp theo như Chain-of-Experts và OptiMUS phân rã vai trò formulator, coder, evaluator và debugger; OptiMUS còn dùng pipeline mô-đun để xử lý mô tả dài (Ahmaditeshnizi et al. 2024; Xiao et al. 2024). Autoformulation xem không gian formulation như một cây tìm kiếm và dùng MCTS để khám phá các giả thuyết (Astorga et al. 2024).

Ưu điểm của nhóm này là triển khai không cần huấn luyện lại; điểm yếu là feedback chủ yếu đến từ code execution, solver status hoặc LLM tự đánh giá. Các tín hiệu này cải thiện executability nhưng chưa tạo certificate ngữ nghĩa.

3.2 Giai đoạn 2: Dữ liệu có cấu trúc và post-training có solver

ORLM, LLMOPT, OptMATH, Step-Opt và OptiTrust cải thiện chất lượng bằng instruction tuning, structured targets và dữ liệu tổng hợp có kiểm tra (Huang et al. 2025; Jiang et al. 2024; Lima et al. 2025; Lu et al. 2025; Wu et al. 2025). SIRL đưa syntax, feasibility, objective và thống kê từ tập LP vào reward RLVR; EVOM dùng outcome-only execution reward và hỗ trợ đổi solver backend (Chen et al. 2025; Guan et al. 2026).

Đây là bước tiến quan trọng ở *generation quality*. Tuy nhiên, reward của SIRL vẫn dùng optimal objective y^* trong dữ liệu huấn luyện, nên không tự giải quyết oracle problem khi triển khai trên một yêu cầu mới không có nhãn (Chen et al. 2025). Solver chỉ xác nhận mô hình được sinh đã được giải đúng theo nội dung của chính nó.

3.3 Giai đoạn 3: Intermediate representation và validation theo stage

LLMOPT dùng five-element formulation, còn CIR/R2C tách rule logic khỏi mathematical instantiation bằng constraint archetype và modeling paradigm (Jiang et al. 2024; Lyu et al. 2026). TriVAL đặt cổng construct-validate-revise ở ba stage: semantic specification, mathematical formulation và code (Fang et al. 2026). ConstraintLLM bổ sung retrieval theo ràng buộc và guided self-correction cho constraint programming (Shi et al. 2025).

IR giúp type checking, provenance và deterministic compilation. Dù vậy, một IR được sinh sai nhưng nội bộ nhất quán vẫn có thể đi qua toàn bộ pipeline. VeriSimpl quan sát đúng failure này: formulation và LLM reasoning có thể đồng thuận trên cùng một cách hiểu sai (Rahman et al. 2026).

3.4 Giai đoạn 4: Verification hướng tới silent failure

Đây là frontier liên quan trực tiếp nhất đến đề tài:

- **Component-level evaluation** đo precision/recall của variable và constraint, RMSE của hệ số, thay vì chỉ outcome accuracy (Refai and Ahmed 2025).
- **Mutation/testing**: một nhóm agent sinh testing API, test và mutation để đo fault-detection power (Zadorojniy et al. 2025).
- **Behavioral verification**: ReLoop perturb tham số, quan sát phản ứng của optimum và dùng diagnosis-guided repair (Lian et al. 2026).
- **Multi-stage hoặc dual-side verification**: TriVAL kiểm ở ba stage; Opt-Verifier kiểm từ phía structure và solution (Fang et al. 2026; Liu et al. 2026).

- **Solver simplification:** VeriSimpl cố định/mask biến và ràng buộc để tạo câu hỏi chặn đoán đơn giản hơn cho LLM (Rahman et al. 2026).
- **Taxonomy-grounded auditing:** OptArgus dùng các specialist auditor cho objective, variable, constraint và implementation errors (Z. Li et al. 2026a).
- **Sound falsification:** Li và Hai suy ra test từ duality, comparative statics và polyhedral limits; khi một test sound bị vi phạm, đó là certificate rằng candidate không trung thành với role đã khai báo (H. Li and Hai 2026).

3.5 Ma trận các công trình tiêu biểu

Bảng 2: Ma trận SOTA theo cơ chế đảm bảo. “Bất silent failure” chỉ phản ánh phạm vi được paper kiểm chứng, không phải bảo đảm đầy đủ.

Công trình	Trạng thái	Cơ chế chính	Bất silent failure?	Cần reference lúc deploy?	Giới hạn liên quan
OptiMUS (Ahmaditeshnizi et al. 2024)	Đã phân biện	Agent mô-đun, solver, debug	Một phần	Không để chạy; cần nhân để đánh giá	Feedback execution không chứng minh semantics.
ORLM (Huang et al. 2025)	Đã phân biện	Dữ liệu tổng hợp, SFT, IndustryOR	Một phần	Có trong đánh giá	Tăng generation accuracy; không phải deployment verifier.
SIRL (Chen et al. 2025)	Đã phân biện	RLVR với solver và LP features	Một phần	Có y^* trong reward và evaluation	Có thể reward đúng outcome nhưng sai formulation.
Step-Opt (Wu et al. 2025)	Đã phân biện	Iterative synthesis, structured validation	Một phần	Có trong lọc dữ liệu/evaluation	Validation chủ yếu ở data/training.
CIR/R2C (Lyu et al. 2026)	Preprint	Typed IR, retrieval, multi-agent	Một phần	Không bắt buộc	IR sai nhưng nhất quán vẫn tồn tại.
TriVAL (Fang et al. 2026)	Preprint	Ba cổng validation theo stage	Có	Không bắt buộc	Validator dựa nhiều vào LLM; lỗi tương quan.
ReLoop (Lian et al. 2026)	Preprint	Perturbation, IIS, repair guard	Có	Không	Threshold heuristic; hạn chế với sai hệ số/equivalence.
Opt-Verifier (Liu et al. 2026)	Đã phân biện	Structure-side + solution-side	Có	Không bắt buộc	Chất lượng phụ thuộc verifier reasoning.
VeriSimpl (Rahman et al. 2026)	Đã phân biện	Solver-generated simplifications	Có	Không	Có thể cùng hiểu sai variable hoặc requirement bị bỏ sót.
OptArgus (Z. Li et al. 2026a)	Preprint	Taxonomy, specialist auditors, abstention	Có	Không bắt buộc	Detection, chưa đánh giá closed-loop repair.
Agent validation (Zadorojniy et al. 2025)	Preprint	Testing API + tests + mutations	Có	Không bắt buộc	Mutation coverage không đồng nghĩa semantic correctness.

Công trình	Trạng thái	Cơ chế chính	Bất silent failure?	Cần reference lúc deploy?	Giới hạn liên quan
Falsification battery (H. Li and Hai 2026)	Preprint	Sound solver tests, detection limits	Có	Không	Detection chứ chưa repair; blind set còn tồn tại.
PEARL (Lu et al. 2026)	Preprint	Learned solve–debug–revise tool loop	Một phần	Không bắt buộc	Verified solve rate chưa phải certificate semantics.

4 Những kết luận đã đủ mạnh để làm nền cho paper

1. **Executability và feasibility là điều kiện cần, không đủ.** Solver không biết ý định trong natural language; nó chỉ giải mô hình được cung cấp (Lian et al. 2026).
2. **Khớp objective trên một instance không định danh formulation.** OptArgus xây dựng trực tiếp bài toán phát hiện artifact sai cấu trúc dù outcome có vẻ đúng (Z. Li et al. 2026a).
3. **Verifier dùng cùng LLM có common-mode failure.** ReLoop thừa nhận constraint extraction và generation dùng cùng model; VeriSimpl ghi nhận cả solver formulation và reasoning có thể nhất quán với cùng một hiểu sai (Lian et al. 2026; Rahman et al. 2026).
4. **Reference-free verification chỉ có thể falsify, không chứng minh completeness tổng quát.** Một sound battery có thể chứng nhận candidate sai khi test fail; khi mọi test pass, candidate vẫn có thể thuộc blind set (H. Li and Hai 2026).
5. **Benchmark noise là biến nhiễu lớn.** Một survey do chuyên gia kiểm tra báo cáo nhiều benchmark có lỗi mô tả, tham số hoặc ground truth; MIPLIB-NL được tạo để đưa đánh giá tới mô hình công nghiệp có quy mô lớn hơn (Z. Li et al. 2026b; Xiao et al. 2025).

Giới hạn cần ghi rõ

Không nên viết claim “đảm bảo formulation đúng hoàn toàn” nếu hệ thống chỉ chạy test hữu hạn. Claim đúng hơn là một trong ba loại: (i) certificate về *unfaithfulness* khi vi phạm điều kiện cần; (ii) bảo đảm không regression trên tập test đã quan sát; hoặc (iii) risk control thống kê dưới giả định exchangeability.

5 Khoảng trống nghiên cứu còn mở

G1+G2 tạo thành một câu chuyện paper gọn và mới hơn: *certificate không chỉ dùng để phát hiện, mà điều khiển minimal repair; còn khi certificate im lặng, một selective gate được hiệu chỉnh để tránh overclaim.*

6 Đề xuất chính: CertOpt

Đề xuất trọng tâm

CertOpt là một pipeline training-free hoặc light-training cho LP/MILP. Hệ thống sinh model trong Requirement-Traceable IR, chạy một tập verifier độc lập, chuyển failure thành certificate có provenance, sửa đúng program slice liên quan, chạy regression trên toàn bộ lịch sử certificate, rồi dùng risk-calibrated gate để chọn accept, repair hoặc abstain/escalate.

Bảng 3: Research gaps sau khi tính đến các paper mới nhất đến 31/07/2026.

ID	Khoảng trống	Bảng chứng/động lực	Khả thi
G1	Certificate → localized repair : các verifier mạnh nhất hoặc chỉ detect, hoặc repair bằng prompt heuristic. Chưa rõ certificate nào nên sửa node nào và làm sao tránh phá phần đang đúng.	Falsification paper chỉ đánh giá detection; OptArgus chưa repair; ReLoop còn regression threshold và failure với lỗi cấu trúc lớn (H. Li and Hai 2026; Z. Li et al. 2026a; Lian et al. 2026).	Cao
G2	Accept/repair/abstain có risk control : “không thấy lỗi” đang bị dùng như “đúng”, dù verifier có blind set.	Detection limits chứng minh blind set không rộng; human-in-the-loop cần biết lúc nào phải can thiệp (H. Li and Hai 2026; Xiao et al. 2025).	Cao
G3	Multi-error và OOD verification : benchmark injected error thường đơn lỗi; lỗi tự nhiên chồng lấp và khó định vị.	OptArgus nêu rõ controlled set dùng một hallucination mỗi case và chưa bao phủ tương tác đa lỗi (Z. Li et al. 2026a).	Cao
G4	Độc lập giữa spec extraction và verifier : nhiều pipeline cùng phụ thuộc một LLM, nên sai semantics từ đầu có thể lan qua mọi stage.	Hạn chế được báo cáo bởi ReLoop và VeriSimpl (Lian et al. 2026; Rahman et al. 2026).	Trung bình
G5	Sound tests ngoài MILP : nonlinear, stochastic, robust và multi-objective chưa có catalog điều kiện cần tương ứng.	Falsification battery giới hạn ở MILP/assertion vocabulary; OptArgus cũng chưa đại diện đầy đủ các lớp này (H. Li and Hai 2026; Z. Li et al. 2026a).	Thấp–trung bình

6.1 Định nghĩa bài toán

Từ mô tả p , bộ trích xuất sinh Requirement IR

$$R = \{r_i = (\text{id}_i, \text{entity}_i, \text{type}_i, \text{sense}_i, \text{value}_i, \text{source-span}_i)\}_{i=1}^m.$$

Generator tạo candidate M_0 trong một canonical IR. Mỗi variable, objective term và constraint của M_0 phải trở về ít nhất một id_i . Compiler C là deterministic:

$$C(M) \longrightarrow \{\text{solver code, LP/MPS artifact, symbol table}\}.$$

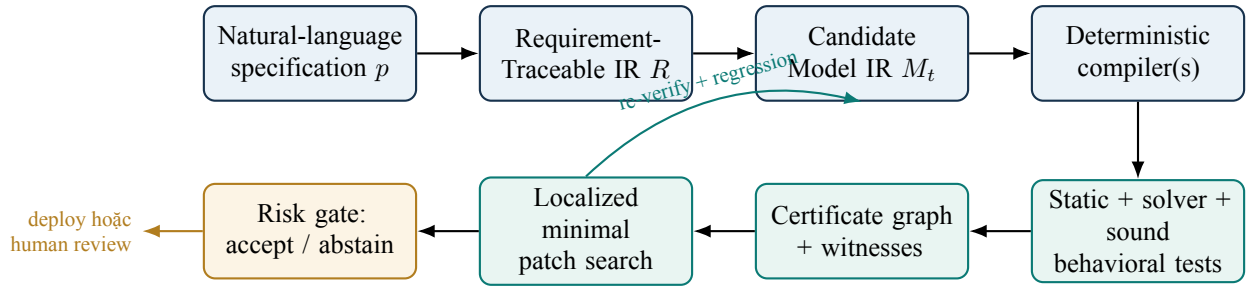
Thiết kế này tách lỗi formulation khỏi lỗi chép code; code không còn được LLM viết tự do sau khi IR đã được chấp nhận.

Một verifier v_j trả về

$$v_j(p, R, M) = \begin{cases} c_j = (\text{type, requirement IDs, model slice, witness}), & \text{nếu phát hiện lỗi,} \\ \emptyset, & \text{nếu không phát hiện.} \end{cases}$$

Với verifier sound, $c_j \neq \emptyset$ là certificate rằng M vi phạm một điều kiện cần trong phạm vi giả định của v_j . Ngược lại, $c_j = \emptyset$ không chứng minh M đúng.

6.2 Kiến trúc



Hình 1: Kiến trúc đề xuất của **CertOpt**. LLM được giới hạn ở extraction, generation và patch proposal; compilation và phần lớn verification là deterministic.

6.3 Bốn thành phần tạo novelty

6.3.1 Requirement-Traceable IR

Mỗi fragment mô hình mang provenance tới câu/phrase nguồn. Hệ thống áp dụng các check:

- **Coverage**: requirement nào chưa có node tương ứng?
- **Uniqueness**: một requirement có bị materialize hai lần không?
- **Type/sense**: count variable có bị continuous; “at least” có bị đảo dấu?
- **Binding**: số liệu trong source span có đi vào đúng entity/constraint?
- **Compiler consistency**: hai backend từ cùng IR có tạo cùng objective, domains và số hàng/cột không?

Khác với một IR chỉ để generation, IR ở đây là *đơn vị audit và repair*. Mục tiêu không phải tuyên bố extraction luôn đúng, mà làm sai lệch trở nên định vị được và có thể escalated.

6.3.2 Certificate graph

Kết hợp bốn nguồn bằng chứng:

1. static schema/type/provenance checks;
2. execution, infeasibility/IIS, unbounded ray và numerical diagnostics;
3. sound falsification tests theo direction, curvature, crush, prohibitive limit, annihilation và exchange (H. Li and Hai 2026);
4. LLM auditor hoặc simplification queries chỉ đóng vai trò *supporting evidence*, không được gọi là proof (Z. Li et al. 2026a; Rahman et al. 2026).

Các certificate được gom thành đồ thị hai phía giữa requirement IDs và model nodes. Điều này trả lời trực tiếp câu hỏi “lỗi nào liên quan tới phần nào”, thay vì đưa toàn bộ code và một traceback dài cho LLM.

6.3.3 Localized minimal repair

Với certificate set Γ_t , chỉ program slice $S(\Gamma_t)$ được mở để chỉnh sửa. Generator sinh K patch candidates $M_t^{(1)}, \dots, M_t^{(K)}$. Candidate được xếp hạng theo:

$$J(M') = \lambda_1 N_{\text{cert}}(M') + \lambda_2 N_{\text{reg}}(M') + \lambda_3 d_{\text{edit}}(M', M_t) + \lambda_4 \text{cost}(M'),$$

với thứ tự ưu tiên dạng lexicographic: loại certificate hiện tại, không làm tái xuất hiện certificate cũ, rồi mới tối thiểu hoá edit và solver cost.

Regression memory lưu toàn bộ witness/test đã pass hoặc từng fail. Một patch chỉ được chấp nhận nếu:

$$\Gamma_{\text{history}}(M') = \emptyset \quad \text{và} \quad \text{StaticInvariant}_{\bar{S}}(M', M_t) = \text{true},$$

nghĩa là không tái phạm lỗi đã chứng kiến và không thay node ngoài repair slice. Đây là guarantee trên test history và syntactic locality, không phải chứng minh semantic equivalence toàn cục.

6.3.4 Calibrated accept–repair–abstain

Sau khi không còn certificate, hệ thống vẫn không được gọi candidate là “verified correct”. Ta xây reliability score $s(M)$ từ:

- disagreement giữa các lần requirement extraction;
- số/loại test có precondition hợp lệ và coverage đạt được;
- bất đồng giữa verifier families;
- repair instability và số vòng lặp;
- cross-solver/compiler disagreement;
- độ mới/OOD của requirement graph so với calibration set.

Trên calibration set có nhãn semantic error, chọn threshold τ để tối đa hoá coverage với ràng buộc risk của accepted set không vượt α . Có thể dùng conformal risk control (Angelopoulos et al. 2022) hoặc confidence-interval calibration cho selective answering (Dong and Shinnou 2026). Claim phải đi kèm giả định exchangeability và evaluation under shift.

6.4 Thuật toán ở mức paper

1. Trích xuất R nhiều lần hoặc bằng hai parser khác nhau; nếu disagreement vượt ngưỡng, yêu cầu làm rõ hoặc gắn cờ uncertainty.
2. Sinh M_0 trong canonical IR và compile deterministic sang solver artifact.
3. Chạy verifier suite; chuyển mọi failure thành Γ_0 .
4. Nếu $\Gamma_t \neq \emptyset$, dựng repair slice, sinh K minimal patches và chọn patch tốt nhất theo J .
5. Re-run toàn bộ historical tests. Roll back nếu có regression.
6. Lặp đến khi hết certificate, hết budget hoặc không có patch cải thiện.
7. Tính $s(M_t)$; **accept** nếu thỏa risk gate, ngược lại **abstain/escalate**. Không cưỡng ép sinh câu trả lời khi bằng chứng yếu.

6.5 Đóng góp có thể claim

Nếu thí nghiệm thành công, paper có thể claim bốn đóng góp sau:

1. một formulation IR có requirement provenance, deterministic compilation và repair slices;
2. một thuật toán certificate-guided minimal repair với regression memory;
3. một selective deployment gate kiểm soát empirical/finite-sample risk thay vì dùng “all tests pass” như certificate correctness;
4. một benchmark protocol đa lỗi, natural errors và operator-held-out OOD để đánh giá detection–repair–abstention end-to-end.

7 Câu hỏi nghiên cứu và giả thuyết

RQ1 – Repair efficacy.

Certificate-guided localized repair có tăng semantic repair success so với regenerate, self-refine, ReLoop-style repair và full-code patching không?

H1.

CertOpt tăng repair success và giảm edit distance, đặc biệt trên localized single/multi-error cases.

RQ2 – Non-regression.

Regression memory và program slicing có giảm tỷ lệ “sửa một lỗi, tạo lỗi khác” không?

H2.

Historical re-verification giảm đáng kể regression rate so với prompt repair không có guard.

RQ3 – Selective reliability.

Risk-calibrated gate có kiểm soát accepted-error rate ở mức mục tiêu trong in-distribution và suy giảm thế nào dưới shift?

H3.

Gate giảm selective risk so với confidence tự báo cáo, self-consistency và “all tests pass”, với trade-off coverage đo được.

RQ4 – Generalization.

Certificate-to-slice mapping có chuyển sang model family, solver backend và error operator chưa thấy không?

H4.

Mapping dựa trên requirement/model graph tổng quát hơn patch template phụ thuộc solver code.

8 Thiết kế thí nghiệm

8.1 Datasets

Bảng 4: Bộ dữ liệu khuyến nghị. Chỉ dùng split đã được làm sạch hoặc tự audit lại.

Dataset	Vai trò	Điểm mạnh	Lưu ý
NL4Opt-clean	Calibration + basic LP	Có logical form/reference, dễ tạo mutation	Quá đơn giản nếu dùng một mình (Ramamonjison et al. 2023; Xiao et al. 2025).
MAMO/IndustryOR-clean	Complex concrete modeling	Dùng rộng trong ORLM/SIRL/ReLoop	Phải dùng bản corrected và công bố exclusion rules (Huang et al. 2025; Xiao et al. 2025).
RetailOpt-190	Compositional stress test	Nhiều ràng buộc tương tác, silent failure rõ	Miền retail; cần split theo family (Lian et al. 2026).
NL4COP	Combinatorial OOD	150 instances, 50 problem types	Kiểm tra khả năng vượt LP textbook (Fang et al. 2026).
MIPLIB-NL	Industrial scale	Grounded trong MIPLIB, 10^3 – 10^6 + variables/constraints	Chi phí solver và IR comparison lớn (Z. Li et al. 2026b).
OptArgus artifacts	Detection/repair benchmark	Clean, injected và natural artifacts; taxonomy chi tiết	Controlled subset chủ yếu single-error (Z. Li et al. 2026a).

8.2 Benchmark lỗi đề xuất

Từ mỗi clean reference M^* , tạo ba tầng:

1. **Single-error controlled:** flip sense; omit constraint or objective term; wrong domain; wrong coefficient or entity; index truncation; incorrect Big- M ; reporting error.
2. **Multi-error compositions:** lấy 2–4 mutation từ các family khác nhau, kiểm tra tương tác và repair order.
3. **Natural errors:** thu output từ ít nhất ba model family và nhiều random seeds; chuyên gia gán nhãn một stratified subset.

Split phải theo *problem template/family*, không random theo instance. Thêm operator-held-out split: một số mutation operators chỉ xuất hiện ở test để đo generalization thật.

8.3 Ground truth và oracle

Không dùng một objective value làm oracle duy nhất. Semantic correctness label nên kết hợp:

- variable/domain và constraint component matching;
- graph/canonical-IR comparison sau khi chuẩn hoá permutation/scaling hợp lệ;
- multi-instance behavioral agreement trên parametric family;
- solver-generated counterexamples khi hai feasible sets khác nhau trong miền hữu hạn;
- OR-expert review cho disagreement và natural artifacts.

Mỗi loại chỉ là một góc nhìn; báo cáo agreement và adjudication protocol. Với MIPLIB-NL lớn, có thể dùng structural reconstruction checks thay cho equivalence đầy đủ.

8.4 Baselines

1. direct generation và structured CoT;
2. full regeneration sau execution error;
3. self-refine/full-code repair;
4. ReLoop behavioral verification + repair (Lian et al. 2026);
5. VeriSimpl (Rahman et al. 2026);
6. TriVAL hoặc Opt-Verifier nếu code/license cho phép (Fang et al. 2026; Liu et al. 2026);
7. OptArgus detection + generic repair prompt (Z. Li et al. 2026a);
8. sound falsification battery không repair (H. Li and Hai 2026);
9. **CertOpt** không risk gate và **CertOpt** đầy đủ.

8.5 Metrics

Generation. Compile rate, solve rate, objective agreement và strict semantic accuracy. Hai metric đầu chỉ là diagnostics.

Detection/localization. Artifact-level AUROC/AUPRC; false-positive rate trên clean models; top- k localization accuracy; certificate coverage theo error type.

Repair.

$$\text{RepairSuccess} = \frac{\#\{\text{incorrect} \rightarrow \text{semantically correct}\}}{\#\{\text{incorrect inputs}\}},$$
$$\text{RegressionRate} = \frac{\#\{\text{một lỗi được sửa nhưng tạo lỗi mới}\}}{\#\{\text{attempted repairs}\}}.$$

Bổ sung edit distance, số vòng, token cost, solver calls và wall-clock latency.

Selective reliability. Coverage, accepted-set error rate, risk-coverage curve, area under risk-coverage curve, và violation rate của target α . Báo cáo riêng in-distribution, operator-held-out, model-held-out và dataset-held-out.

Thống kê. Dùng paired bootstrap confidence interval cho accuracy/repair delta, McNemar test cho paired binary outcomes, và stratified reporting theo error family. Công bố seed, model version, solver version, tolerance và timeout.

8.6 Ablation bắt buộc

9 Kết quả nào sẽ làm paper thuyết phục?

Paper không cần thắng mọi benchmark về raw accuracy. Một kết quả mạnh hơn là:

Bảng 5: Ablation giúp xác định đóng góp thực sự.

Bổ thành phần	Câu hỏi trả lời	Metric nhạy nhất
Không Requirement provenance	Traceability có thật sự giúp localization?	Top- k localization, repair edit distance
Không program slicing	Local repair có tốt hơn full-code rewrite?	Regression rate, token cost
Không historical regression memory	Guard có ngăn tái phạm không?	Regression rate
Chỉ LLM verifier	Solver/sound tests thêm giá trị gì?	FPR, OOD detection
Chỉ sound battery	Supporting verifier có tăng recall không?	Recall với lỗi nằm trong blind set
Không calibrated gate	Calibration có tốt hơn heuristic threshold?	Risk-coverage, target-risk violations
Một compiler/solver	Differential backend có bất implementation mismatch?	Code-level error detection

- cùng semantic accuracy nhưng regression thấp hơn rõ rệt;
- phát hiện/lokalize tốt hơn trên multi-error và operator-held-out;
- đạt target accepted-error rate với coverage hữu ích;
- certificate giúp giảm số token và số vòng repair;
- phân tích failure cho thấy chính xác khi nào sound verifier im lặng và gate abstain.

Một negative result cũng có giá trị: nếu calibration vỡ mạnh dưới dataset shift, paper có thể chỉ ra điều kiện mà selective risk control không chuyển miền và đề xuất group/domain-conditional calibration.

10 Phạm vi MVP và các hướng thay thế

10.1 MVP được khuyến nghị

- Chỉ LP/MILP.
- Hai dataset sạch + RetailOpt-190; MIPLIB-NL là stress test tùy tài nguyên.
- Một open model 7B/8B và một model mạnh hơn.
- HiGHS/SCIP hoặc Gurobi nếu có license; deterministic IR compiler.
- 8–12 mutation operators, sau đó multi-error composition.
- Training-free repair; chỉ calibration layer cần labeled holdout.

MVP này đủ cho một paper systems/empirical nếu benchmark, evaluation và ablation chặt. Không nên đồng thời fine-tune generator, xây benchmark lớn, chứng minh nonlinear theory và triển khai human study trong paper đầu.

10.2 Hướng thay thế A: Nonlinear/Stochastic Falsification

Mở rộng catalog sound tests sang convex nonlinear, stochastic hoặc robust optimization. Đây là hướng có novelty lý thuyết cao: dùng value-function monotonicity, convex duality, KKT, uncertainty-set inclusion hoặc Pareto dominance để tạo necessary-condition tests. Tuy nhiên cần định nghĩa assertion vocabulary mới và chứng minh soundness/detection limits; phù hợp nếu nhóm mạnh về optimization theory.

10.3 Hướng thay thế B: Ambiguity-Aware Interactive Modeling

Thay vì sửa sau khi mô hình sai, hệ thống phát hiện underspecification và hỏi câu hỏi có expected information gain cao nhất. Đánh giá số câu hỏi, semantic accuracy sau đối thoại và human burden. Hướng này bám human-in-the-loop gap (Xiao et al. 2025), nhưng cần simulator hoặc user study đáng tin.

10.4 Hướng thay thế C: Certificate-Derived RL Reward

Biến certificate type và localized witness thành dense RL reward, thay vì chỉ syntax/feasibility/objective. So sánh với SIRL và EVOM (Chen et al. 2025; Guan et al. 2026). Hướng này có tiềm năng lớn nhưng compute cao và dễ reward hacking; nên làm sau khi verifier/repair benchmark đã ổn.

11 Rủi ro và cách giảm thiểu

Bảng 6: Rủi ro chính của đề xuất.

Rủi ro	Giảm thiểu
Requirement extraction sai từ đầu	Multi-parser disagreement; source-span provenance; human escalation; không gọi extraction là ground truth.
Patch “pass tests” nhưng sai ngữ nghĩa	Operator-held-out/natural errors; supporting auditors; accepted-risk calibration; report blind cases.
Calibration không chuyển miền	Dataset-held-out test; group conditional thresholds; báo coverage giảm thay vì ép accept.
So sánh baseline không công bằng	Cùng base model, solver, timeout, sample budget và repaired benchmark; công bố prompt/code.
Benchmark mutations quá nhân tạo	Kết hợp natural artifacts và expert annotation; báo riêng controlled/natural.
IR làm mất modeling strategy hợp lệ	Cho phép nhiều canonical archetypes; semantic normalization; không ép một formulation duy nhất.
Chi phí solver tăng	Budgeted test selection; cache probes; ưu tiên test theo predicted information gain.

12 Lộ trình 12 tuần

Tuần 1–2 Chốt schema Requirement IR/Model IR; dựng compiler tới một solver; tái lập direct/structured baseline.

Tuần 3–4 Tích hợp static checks và falsification battery; chuẩn hoá certificate format; unit test trên hand-crafted errors.

Tuần 5–6 Xây mutation operators, multi-error composer và ground-truth evaluator; audit split dữ liệu.

Tuần 7–8 Cài localized patch search, regression memory và rollback; chạy pilot để điều chỉnh repair budget.

Tuần 9 Xây reliability score và calibration gate; freeze calibration split.

Tuần 10 Main experiments, model-held-out và operator-held-out; thống kê.

Tuần 11 Ablation, error analysis, cost analysis và visualizations.

Tuần 12 Viết paper, artifact checklist và reproducibility package.

13 Khung paper có thể dùng ngay

1. **Introduction:** feasibility–correctness gap; verifier silence không phải correctness; certificate chưa được khai thác có hệ thống cho repair và abstention.
2. **Related Work:** generation/post-training; IR; verification; repair; selective risk.
3. **Problem Setup:** requirement traceability, semantic error, sound certificate, selective policy.
4. **CertOpt:** IR, certificate graph, localized repair, regression memory, risk gate.
5. **Benchmark and Protocol:** cleaned data, multi-error, OOD splits, human adjudication.
6. **Experiments:** RQ1–RQ4, baselines, metrics và statistical tests.
7. **Analysis:** certificate coverage, blind set, calibration under shift, cost.
8. **Limitations:** MILP scope, extraction errors, finite test coverage, exchangeability.
9. **Conclusion:** từ “generate and hope” sang “detect, repair, or abstain with evidence”.

14 Quyết định nghiên cứu được khuyến nghị

Kết luận chính

Research question cốt lõi: “Can sound or high-precision failure certificates be converted into localized, regression-safe repairs, and can the residual risk of verifier silence be controlled by selective abstention?”

Phạm vi tốt nhất cho paper đầu: LP/MILP, training-free inference, requirement-traceable IR, multi-error benchmark và calibrated accept–repair–abstain.

Điểm khác biệt cần bảo vệ: ReLoop đã có behavioral repair; OptArgus đã có taxonomy và detector; falsification-based verification đã có sound tests. Novelty của bạn nằm ở *certificate-to-edit localization + historical regression protection + calibrated selective deployment*, được đánh giá trên multi-error/OOD protocol.

Tài liệu tham khảo

- Ahmaditeshnizi, A., W. Gao, and M. Udell (2024). “OptiMUS: Scalable Optimization Modeling with (MI)LP Solvers and Large Language Models”. In: *Proceedings of the 41st International Conference on Machine Learning*. Vol. 235. Proceedings of Machine Learning Research. PMLR, pp. 577–596. URL: <https://proceedings.mlr.press/v235/ahmaditeshnizi24a.html>.
- Angelopoulos, A. N., S. Bates, A. Fisch, L. Lei, and T. Schuster (2022). “Conformal Risk Control”. In: *arXiv preprint arXiv:2208.02814*. DOI: [10.48550/arXiv.2208.02814](https://arxiv.org/abs/2208.02814). URL: <https://arxiv.org/abs/2208.02814>.
- Astorga, N., T. Liu, Y. Xiao, and M. van der Schaar (2024). “Autoformulation of Mathematical Optimization Models Using LLMs”. In: *arXiv preprint arXiv:2411.01679*. DOI: [10.48550/arXiv.2411.01679](https://arxiv.org/abs/2411.01679). URL: <https://arxiv.org/abs/2411.01679>.
- Chen, Y., J. Xia, S. Shao, D. Ge, and Y. Ye (2025). “Solver-Informed RL: Grounding Large Language Models for Authentic Optimization Modeling”. In: *Advances in Neural Information Processing Systems* 38. URL: https://proceedings.neurips.cc/paper_files/paper/2025/hash/98555b92e5dedbdad5921049281995bc-Abstract-Conference.html.
- Dong, S. and H. Shinnou (2026). “Uncertainty-Aware Abstention in Large Language Models with Provable Alignment Guarantees”. In: *arXiv preprint arXiv:2607.04430*. DOI: [10.48550/arXiv.2607.04430](https://arxiv.org/abs/2607.04430). URL: <https://arxiv.org/abs/2607.04430>.

- Fang, Z., J. Wang, J. Zhong, and Y.-S. Ong (2026). “TriVAL: A Tri-Validation Framework for Faithful Automatic Optimization Modeling”. In: *arXiv preprint arXiv:2605.23966*. DOI: [10.48550/arXiv.2605.23966](https://arxiv.org/abs/2605.23966). URL: <https://arxiv.org/abs/2605.23966>.
- Guan, R., X. Shen, J. Zhang, Y. Zhang, J. Cheng, and R. Xia (2026). “Execution-Verified Reinforcement Learning for Optimization Modeling”. In: *arXiv preprint arXiv:2604.00442*. DOI: [10.48550/arXiv.2604.00442](https://arxiv.org/abs/2604.00442). URL: <https://arxiv.org/abs/2604.00442>.
- Huang, C., Z. Tang, S. Hu, R. Jiang, X. Zheng, D. Ge, B. Wang, and Z. Wang (2025). “ORLM: A Customizable Framework in Training Large Models for Automated Optimization Modeling”. In: *Operations Research*. Published online ahead of print. DOI: [10.1287/opre.2024.1233](https://arxiv.org/abs/2405.17743). URL: <https://arxiv.org/abs/2405.17743>.
- Jiang, C., X. Shu, H. Qian, X. Lu, J. Zhou, A. Zhou, and Y. Yu (2024). “LLMOPT: Learning to Define and Solve General Optimization Problems from Scratch”. In: *arXiv preprint arXiv:2410.13213*. DOI: [10.48550/arXiv.2410.13213](https://arxiv.org/abs/2410.13213). URL: <https://arxiv.org/abs/2410.13213>.
- Li, H. and M. Hai (2026). “Falsification-Based Verification of LLM-Generated Optimization Models: Sound Test Batteries and Their Detection Limits”. In: *arXiv preprint arXiv:2607.16646*. DOI: [10.48550/arXiv.2607.16646](https://arxiv.org/abs/2607.16646). URL: <https://arxiv.org/abs/2607.16646>.
- Li, Z., Z. Guo, X. Lu, J. Wang, J. Song, C. Shen, J. Wu, and M. Sun (2026a). “OptArgus: A Multi-Agent System to Detect Hallucinations in LLM-Based Optimization Modeling”. In: *arXiv preprint arXiv:2605.11738*. DOI: [10.48550/arXiv.2605.11738](https://arxiv.org/abs/2605.11738). URL: <https://arxiv.org/abs/2605.11738>.
- Li, Z., H. Lu, T. Wei, Y. Chen, W. Liu, Y. Lan, F. Zhang, and Z. Wen (2026b). “Constructing Industrial-Scale Optimization Modeling Benchmark”. In: *Proceedings of the 43rd International Conference on Machine Learning*. To appear. URL: <https://arxiv.org/abs/2602.10450>.
- Lian, J. J., Y. Sun, H. Chen, C. Zhang, H. Qin, and C.-P. Teo (2026). “ReLoop: Structured Modeling and Behavioral Verification for Reliable LLM-Based Optimization”. In: *arXiv preprint arXiv:2602.15983*. DOI: [10.48550/arXiv.2602.15983](https://arxiv.org/abs/2602.15983). URL: <https://arxiv.org/abs/2602.15983>.
- Lima, V., D. T. Phan, J. Kalagnanam, D. Patel, and N. Zhou (2025). “Toward a Trustworthy Optimization Modeling Agent via Verifiable Synthetic Data Generation”. In: *arXiv preprint arXiv:2508.03117*. DOI: [10.48550/arXiv.2508.03117](https://arxiv.org/abs/2508.03117). URL: <https://arxiv.org/abs/2508.03117>.
- Liu, H., J. Wang, B. Niu, X. Han, Y. Xu, M. Ye, Z. Geng, F. Zhu, T. Zhong, M. Yuan, and J. Hao (2026). “Opt-Verifier: Unleashing the Power of LLMs for Optimization Modeling via Dual-Side Verification”. In: *Proceedings of the 43rd International Conference on Machine Learning*. URL: <https://arxiv.org/abs/2605.29556>.
- Lu, H., Z. Li, Y. Chen, Y. Lan, F. Zhang, and Z. Wen (2026). “PEARL: Solver-in-the-Loop Interactive Optimization Modeling from Natural Language”. In: *arXiv preprint arXiv:2607.18256*. DOI: [10.48550/arXiv.2607.18256](https://arxiv.org/abs/2607.18256). URL: <https://arxiv.org/abs/2607.18256>.
- Lu, H., Z. Xie, Y. Wu, C. Ren, Y. Chen, and Z. Wen (2025). “OptMATH: A Scalable Bidirectional Data Synthesis Framework for Optimization Modeling”. In: *Proceedings of the 42nd International Conference on Machine Learning*. Vol. 267. Proceedings of Machine Learning Research. PMLR, pp. 40769–40802. URL: <https://proceedings.mlr.press/v267/lu25o.html>.
- Lyu, Z., S. Hu, L. Liu, H. Yang, and M. Li (2026). “Canonical Intermediate Representation for LLM-Based Optimization Problem Formulation and Code Generation”. In: *arXiv preprint arXiv:2602.02029*. DOI: [10.48550/arXiv.2602.02029](https://arxiv.org/abs/2602.02029). URL: <https://arxiv.org/abs/2602.02029>.
- Rahman, S. A., S. A. A. Cuellar, G. Raissov, and M. Raza (2026). “VeriSimpl: Robust Optimization Modeling from Natural Language using Simplification-Based Verification”. In: *Proceedings of the 43rd International Conference on Machine Learning*. URL: <https://arxiv.org/abs/2607.20474>.
- Ramamonjison, R., H. Li, T. Yu, S. He, V. Rengan, A. Banitalebi-Dehkordi, Z. Zhou, and Y. Zhang (2022). “Augmenting Operations Research with Auto-Formulation of Optimization Models From Problem Descriptions”. In: *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing: Industry Track*. Association for Computational Linguistics, pp. 29–62. DOI: [10.18653/v1/2022.emnlp-industry.4](https://aclanthology.org/2022.emnlp-industry.4/). URL: <https://aclanthology.org/2022.emnlp-industry.4/>.

- Ramamonjison, R., T. T. Yu, R. Li, H. Li, G. Carenini, B. Ghaddar, S. He, M. Mostajabdaveh, A. Banitalebi-Dehkordi, Z. Zhou, and Y. Zhang (2023). “NL4Opt Competition: Formulating Optimization Problems Based on Their Natural Language Descriptions”. In: *NeurIPS 2022 Competition Track*. Vol. 220. Proceedings of Machine Learning Research. PMLR, pp. 189–203. URL: <https://arxiv.org/abs/2303.08233>.
- Refai, D. and M. Ahmed (2025). “Peering Inside the Black Box: Uncovering LLM Errors in Optimization Modelling through Component-Level Evaluation”. In: *arXiv preprint arXiv:2510.16943*. DOI: [10.48550/arXiv.2510.16943](https://arxiv.org/abs/2510.16943). URL: <https://arxiv.org/abs/2510.16943>.
- Shi, W., M. Liu, W. Zhang, L. Shi, F. Jia, F. Ma, and J. Zhang (2025). “ConstraintLLM: A Neuro-Symbolic Framework for Industrial-Level Constraint Programming”. In: *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, pp. 15999–16019. DOI: [10.18653/v1/2025.emnlp-main.809](https://aclanthology.org/2025.emnlp-main.809/). URL: <https://aclanthology.org/2025.emnlp-main.809/>.
- Wu, Y., Y. Zhang, Y. Wu, Y. Wang, J. Zhang, and J. Cheng (2025). “Training LLMs for Optimization Modeling via Iterative Data Synthesis and Structured Validation”. In: *Findings of the Association for Computational Linguistics: EMNLP 2025*. Association for Computational Linguistics, pp. 12880–12896. DOI: [10.18653/v1/2025.findings-emnlp.691](https://aclanthology.org/2025.findings-emnlp.691/). URL: <https://aclanthology.org/2025.findings-emnlp.691/>.
- Xiao, Z., J. Xie, L. Xu, S. Guan, J. Zhu, X. Han, X. Fu, W. Yu, H. Wu, W. Shi, Q. Kang, J. Duan, T. Zhong, M. Yuan, J. Zeng, Y. Wang, G. Chen, and D. Zhang (2025). “A Survey of Optimization Modeling Meets LLMs: Progress and Future Directions”. In: *arXiv preprint arXiv:2508.10047*. DOI: [10.48550/arXiv.2508.10047](https://arxiv.org/abs/2508.10047). URL: <https://arxiv.org/abs/2508.10047>.
- Xiao, Z., D. Zhang, Y. Wu, L. Xu, Y. J. Wang, X. Han, X. Fu, T. Zhong, J. Zeng, M. Song, and G. Chen (2024). “Chain-of-Experts: When LLMs Meet Complex Operations Research Problems”. In: *The Twelfth International Conference on Learning Representations*. URL: <https://openreview.net/forum?id=HobyL1B9CZ>.
- Zadorojniy, A., S. Wasserkrug, and E. Farchi (2025). “An Agent-Based Framework for the Automatic Validation of Mathematical Optimization Models”. In: *arXiv preprint arXiv:2511.16383*. DOI: [10.48550/arXiv.2511.16383](https://arxiv.org/abs/2511.16383). URL: <https://arxiv.org/abs/2511.16383>.